Totoro Using Quantum supersampling

Pongsakorn Saksakulchai 6914500746 & GPT-6.1-Sol (Coding Assistance)

1. Introduction

This project reconstructs a grayscale image of Totoro with quantum supersampling for the Quantum Programming course. Supersampling estimates a pixel's average brightness from multiple samples within its area. These samples reduce errors caused by choosing a single point, particularly near boundaries and details smaller than a pixel.

For a binary region with N subpixels and M white subpixels, the desired brightness is $a = M/N$. A classical method can count every white subpixel or estimate the fraction from random samples. In Chapter 11 of *Programming Quantum Computers*, quantum supersampling (QSS) flips the phases of white subpixels and estimates their count without reading each subpixel value [1, Ch. 11, pp. 211–228].

Quantum phase estimation (QPE) needs an operator whose eigenphase depends on brightness. The Grover operator gives this relation through $a = \sin^2\theta$, so QPE can estimate M. Quantum counting is the estimation subroutine within QSS. The classical program divides the image into tiles, constructs their oracles, and writes decoded brightness values to the output image.

The book compares QSS noise with Monte Carlo sampling noise. In its examples, QSS produces some exactly correct pixels and some large errors, while Monte Carlo produces small errors across more pixels. These observations describe the book's experiments, which use different conditions from this simulation [1, pp. 211–212, 226–228].

2. Preliminaries

2.1 Grover's algorithm

Grover's algorithm uses amplitude amplification to increase the probability of measuring a marked state by repeating a phase oracle O and a diffusion operator D. Starting from the uniform state $|s\rangle$, define $Q = DO$, where O reverses the amplitudes of marked states and $D = 2|s\rangle\langle s| - I$. Together, these operations rotate the state in the two-dimensional subspace spanned by the marked and unmarked groups. The angle depends on M/N. Quantum counting estimates this angle to recover the number of marked states.

2.2 Quantum Fourier transform

The quantum Fourier transform (QFT) maps periodic phase patterns to computational basis states. For a register of t qubits, let $T = 2^t$. Then $\mathrm{QFT}|k\rangle = T^{-1/2}\sum_{y=0}^{T-1} e^{2\pi iky/T}|y\rangle$. In QPE, the inverse QFT converts phases accumulated on the counter through controlled powers of an operator into a distribution of readout values y. The number of counter qubits limits the phase resolution.

2.3 Quantum phase estimation

For $U|u\rangle = e^{2\pi i\phi}|u\rangle$, QPE prepares a counter register with Hadamard gates, applies controlled-U^{2^j} operations, and performs the inverse QFT. The readout y/T estimates ϕ. When the target is a superposition of eigenstates, the readout distribution reflects their eigenphases. Here, $U = Q$ and the target starts in $|s\rangle$. Decoding the Grover eigenphase gives the white-subpixel fraction.

3. Methodology

3.1 Representing a pixel with a binary mask

Classical preprocessing converts the source image to grayscale and resizes it to 512×512 for a 128×128 output. A 4×4 Bayer dithering pattern produces a binary image. The program divides it into 4×4 tiles, one per output pixel, and estimates each tile's average to downsample the image. Let $f(x) \in \{0, 1\}$ indicate whether a subpixel is white. Basis indices use $x = 4r + c$, where $r, c \in \{0, 1, 2, 3\}$. The count and brightness are

$$M = \sum_{x=0}^{15} f(x), \qquad a = \frac{M}{16}. \tag{1}$$

The classical reference is the mean of this binary mask. The error metric for quantum counting excludes errors from grayscale conversion, resizing, and dithering.

3.2 From the phase oracle to the Grover angle

Four subpixel qubits represent the 16 positions. Hadamard gates prepare the uniform superposition

$$|s\rangle = H^{\otimes 4}|0000\rangle = \frac{1}{4}\sum_{x=0}^{15}|x\rangle. \tag{2}$$

The oracle and Grover operator follow the implementation:

$$\begin{aligned} O_f|x\rangle &= (-1)^{f(x)}|x\rangle, \\ D = 2|s\rangle\langle s| - I, &\qquad Q = DO_f. \end{aligned} \tag{3}$$

White subpixels acquire a phase shift of π, but each basis state still has probability $1/16$ after the oracle. A direct measurement of this register does not reveal the white-subpixel count. Applying Q gives eigenphases that QPE can use to estimate the count.

For $0 < M < 16$, let $|G\rangle$ be the normalized superposition of white basis states and $|B\rangle$ the normalized superposition of the remaining states. Then

$$|s\rangle = \sin\theta|G\rangle + \cos\theta|B\rangle, \qquad \sin^2\theta = \frac{M}{16}. \tag{4}$$

In the basis $(|G\rangle, |B\rangle)$, Q has the representation

$$Q = \begin{pmatrix} \cos 2\theta & \sin 2\theta \\ -\sin 2\theta & \cos 2\theta \end{pmatrix}, \qquad \lambda_\pm = e^{\pm 2i\theta}. \tag{5}$$

Using the convention $e^{2\pi i\phi}$, the eigenphases are $\phi_+ = \theta/\pi$ and $\phi_- = 1 - \theta/\pi$. QPE can therefore estimate the count from these phases. At the endpoints $M = 0$ and $M = 16$, the phase of $|s\rangle$ is 0 and $1/2$, respectively.

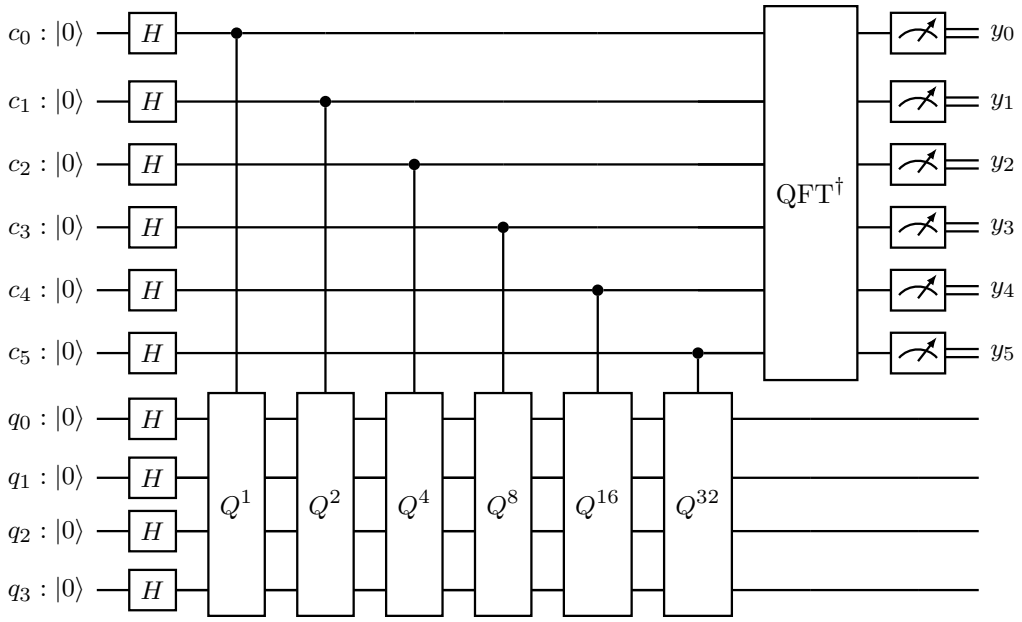


Figure 1. Quantum counting within QSS, built by `build_counting_circuit()`. The counter qubits are $c_0, \ldots, c_5$, and the subpixel qubits are $q_0, \ldots, q_3$. Each c_j controls Q^{2^j} on all four subpixel qubits. Only the counter is measured, with $y = \sum_{j=0}^{5} 2^j y_j$ and c_0 the least significant bit. Statevector simulation uses the unmeasured circuit. Drawn with Quantikz [3].

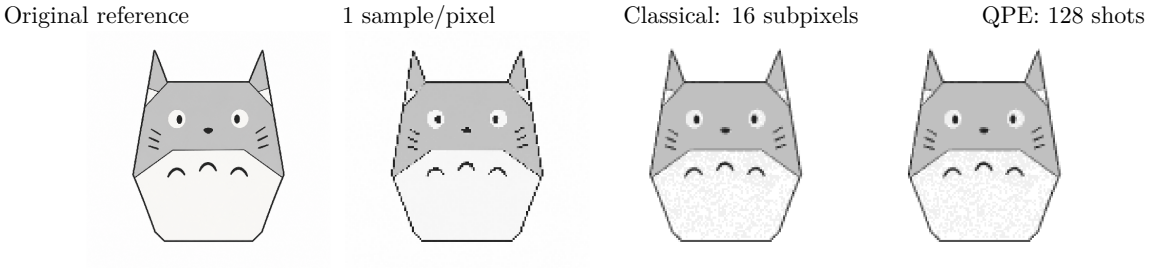


Figure 2. Totoro reconstruction. From left to right: the original Totoro reference, a single sample per pixel, the classical mean of 16 subpixels, and the QPE reconstruction decoded from 128 shots per pixel. The two rightmost panels use the same masks after Bayer dithering. Their agreement measures the accuracy of the estimated mask averages. It does not measure error against the original image.

3.3 Estimating phase and decoding brightness

Figure 1 uses $t = 6$ and $T = 64$. Counter qubit j controls Q^{2^j} for $j = 0, \ldots, 5$. Writing the counter value before the inverse QFT as k, the joint state is

$$\frac{1}{\sqrt{T}} \sum_{k=0}^{T-1} |k\rangle Q^k |s\rangle. \tag{6}$$

For an eigenstate component, Q^k accumulates phase $e^{\pm 2ik\theta}$. The inverse QFT concentrates outcomes near $y \approx T\theta/\pi$ or $y \approx T(1 - \theta/\pi)$. The two peaks arise because $|s\rangle$ is a superposition of eigenstates. Both give the same brightness through

$$\widehat{a} = \sin^2\Big(\frac{\pi y}{T}\Big), \qquad \widehat{M} = 16\widehat{a}. \tag{7}$$

The readout y is not directly the white-subpixel count. A single readout can be inaccurate when the phase lies between counter bins.

The signs of D and Q must match these definitions. Applied alone, Q and $-Q$ differ by a global phase. In controlled-Q, that difference becomes a relative phase between the control branches and can shift the QPE peaks. A lookup table must therefore use the same convention as the circuit.

3.4 Outcome folding and calibration

The code combines outcomes y and $T - y$ into $z = \min(y, T-y)$ without double counting $z = 0$ or $z = T/2$. It then simulates representative masks for $M = 0, 1, \ldots, 16$ and finds each peak $p_M = \arg\max_z P(z \mid M)$. The lookup $L(z)$ selects the count M whose peak p_M is closest to z. With six counter qubits, the recorded peaks for all 17 counts are distinct. Distinct peaks do not imply disjoint distributions.

For each pixel, the program samples 128 shots from the folded distribution, selects the most frequent bin z_{mode}, and assigns brightness $L(z_{\mathrm{mode}})/16$. The decoder uses the mode across multiple shots. It does not apply Eq. (7) to every shot and average the results. The nearest-peak lookup also differs from a maximum-likelihood decoder using the full distributions.

For example, a mask with $M = 4$ white subpixels has $a = 1/4$, giving $\theta = \pi/6$ and phases $1/6$ and $5/6$. At $T = 64$, the peaks are near $y = 11$ and 53, which fold near $z = 11$. Direct substitution of $y = 11$ gives $\widehat{a} \approx 0.264$. Calibration maps this peak back to $M = 4$, producing brightness 0.25.

Table 1. Configuration and existing results from the metrics file.

Quantity	Value
Output image	128×128
Subpixels per pixel	$4 \times 4 = 16$
Counter + subpixel qubits	$6 + 4 = 10$
Shots per pixel / seed	128 / 7
Distinct masks simulated	270
Calibration circuits	17
MAE against mask mean	0
Maximum absolute error	0
Pixels matching the baseline	16,384/16,384

3.5 Simulation scope and circuit cost

Qiskit Statevector computes the counter probabilities, and a classical random generator samples shots with seed 7. The program builds a diagonal oracle matrix from a known mask and implements controlled powers as dense matrices. The book instead uses a quantum pixel shader to compute shapes and flip phases [1, pp. 215–226].

In an oracle-query model, implementing each Q^{2^j} by repeated applications of Q requires $1+2+4+8+16+32 = 63$ Grover iterations per QPE execution. Running 128 shots would therefore require 8,064 iterations per pixel, excluding calibration and state preparation. This query count assumes that each power is expanded into repeated Grover iterations. The simulator instead applies six dense controlled-power gates. Its runtime measures the cost of that matrix implementation.

4. Results

4.1 Recorded results and evaluation metric

Table 1 lists the configuration and results from the saved metrics and figures [2]. The experiment was not rerun during report preparation. Let a_p be the classical mean of the mask for pixel p and $\widehat{a}_p$ the estimate decoded from shots. For $P = 128^2 = 16{,}384$ pixels, the mean absolute error is

$$\mathrm{MAE} = \frac{1}{P}\sum_{p=1}^{P} |\widehat{a}_p - a_p|. \tag{8}$$

The recorded MAE and maximum absolute error are zero. All 16,384 pixels match the baseline. The program evaluates these metrics on brightness values before saving the image as 8-bit grayscale. Figure 2 shows the same agreement between the estimated and classical mask averages.

An earlier self-test run checked all 17 white-subpixel counts and one spatial permutation per count. It recorded a maximum probability difference of approximately 1.11×10^{-16}. These results agree with the expected dependence on the white-subpixel count under uniform preparation and this oracle. The self-test runs with `--self-test`. It does not test finite-shot reconstruction across all random seeds.

4.2 Interpreting zero reconstruction error

The experiment has 17 possible brightness levels. Six counter qubits separate their peaks, and the decoder uses calibration and the mode of 128 shots. The recorded statevector-mode reconstruction also matches the baseline. This agreement holds for the tested image. Individual measurements can still be wrong, and the results do not guarantee zero error for larger subpixel counts.

The classical baseline counts the white entries in the same 16-entry masks. This small problem does not establish a speed or image-quality advantage for QSS. The experiment does not compare QSS with Monte Carlo under an equal oracle-query budget, so it cannot test the book's observations about sampling noise.

The simulation caches distributions for identical masks and evaluates 270 distinct masks. Runtime covers dense linear algebra, caching, sampling, and figure generation. The experiment does not run on quantum hardware or model physical gate and measurement noise.

5. Conclusion

Binary quantum supersampling estimates brightness by counting subpixels marked by phase flips. The Grover eigenphase depends on their fraction, and QPE followed by decoding recovers brightness through $M/16 = \sin^2\theta$. Quantum counting supplies the estimate for each output pixel.

With 10 qubits, calibration, and the mode of 128 shots, the Totoro simulation reproduces the classical mask mean for every pixel in the tested image. Testing the book's noise-related observations requires a comparison with Monte Carlo sampling under comparable oracle-query costs and decoding rules. That comparison must measure error distributions and the fraction of exactly correct pixels.

References

[1] E. R. Johnston, N. Harrigan, and M. Gimeno-Segovia, *Programming Quantum Computers: Essential Algorithms and Code Samples*. O'Reilly Media, 2019, Ch. 11, pp. 211–234. Page citations refer to printed book pages, corresponding to PDF pages 226–249 in the project's reference file.

[2] Internal files in the `quantum-supersampling` project: `quantum_supersampling.py`, `qss_output/metrics.json`, and `qss_output/comparison.png`.

[3] A. Kay, "Tutorial on the Quantikz Package," arXiv:1809.03842, 2018. `https://arxiv.org/abs/1809.03842`.